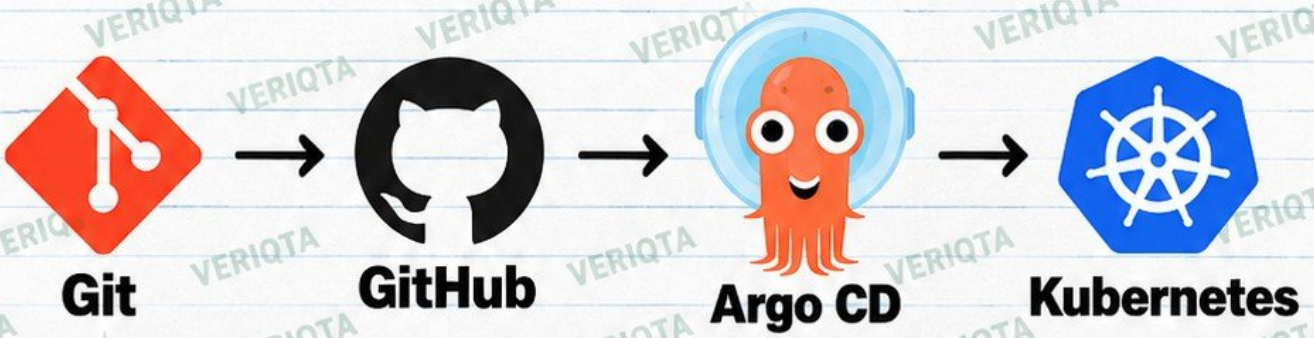
VERIQTA

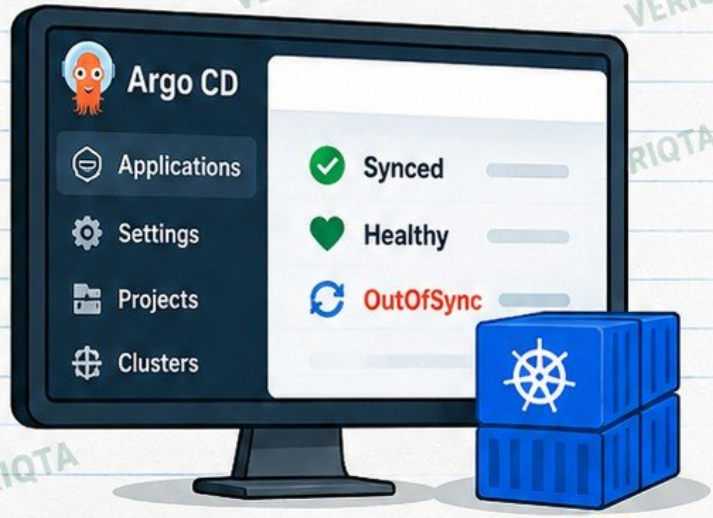
ArgoCD for Beginners:

Your First GitOps Deployment



Code Changes → Git Repository → Auto Sync → Running Application

- Understand GitOps
- Install and Configure Argo CD
- Deploy a Real Application
- Auto Sync and Self-Healing
- Detect and Fix Failures
- Production Best Practices



Instagram @veriqta | GitHub @veriqta | LinkedIn @veriqta | Telegram @veriqta | X @veriqta | Discord @veriqta

1. The Deployment Problem Argo CD Solves

VERIQTA

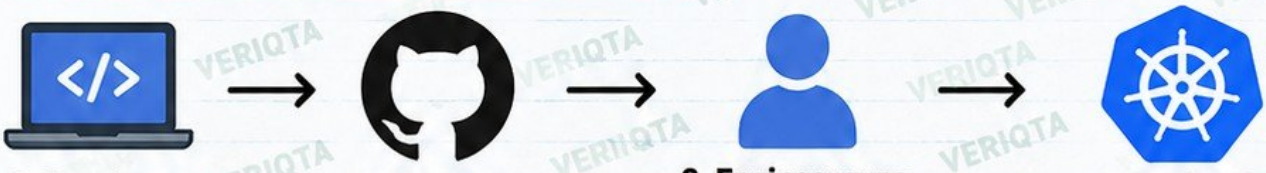
The Real Scenario

A developer merges a change to the application configuration in Git, but production still runs the previous version. This happens when deployments are done manually using kubectl or ad-hoc scripts.

The Problem

Manual deployments are error-prone. They create inconsistent environments, weak audit trails and configuration drift, making it hard to know what is actually running and why.

How Manual Deployment Usually Works



1. Developer makes a change
2. Merge to Git repository
3. Engineer runs kubectl apply manually
4. Application updated in the cluster (if successful)

Where It Can Fail

Change not deployed
Developer merges a change, but no one runs the deployment command.

Wrong cluster or namespace
Engineer applies the change to the wrong environment.

Partial deployment
Some resources are updated, but others are missed.

No clear audit trail
Hard to know who changed what, when and why.

Results of Manual Deployment

- Inconsistent environments (dev, staging, production not matching).
- Configuration drift (cluster state differs from Git).
- Difficult to track changes or roll back.
- Higher risk of human error.
- Teams cannot easily see what is running.
- Slower and less reliable deployments.

What Argo CD Solves

- Uses Git as the source of truth.
- Automatically applies approved changes.
- Detects and fixes configuration drift.
- Provides a clear audit trail of changes.
- Ensures consistent environments.
- Reduces manual errors.
- Makes deployments repeatable and reliable.
- Shows the actual state of your applications.

Practical Focus

Follow one application change from Git to Kubernetes and identify where a manual deployment can fail. This will help you understand why teams use GitOps and how Argo CD improves the process.

2. GitOps Explained Through One Real Deployment

VERIQTA

What is GitOps?

GitOps is a way to manage your infrastructure and application configuration using **Git** as the source of truth. You store your **desired state** in a Git repository, and a tool like **Argo CD** automatically makes the cluster match that state.

Why GitOps Exists

- Reduces manual deployments
- Keeps environments consistent
- Provides a clear audit trail of changes
- Detects and fixes configuration drift
- Makes deployments repeatable and reliable
- Allows teams to work together safely

How GitOps Works (Simple View)



1. Developer changes YAML files
2. Commit and push to Git repository
3. Argo CD watches for changes
4. Cluster is automatically updated to match Git

Desired State vs Live State

Desired State (in Git)

- Stored in Git repository
- Version controlled
- Reviewed through pull requests
- Represents what should be running
- Easier to audit and track

Live State (in Cluster)

- Actual resources running in Kubernetes
- Can be changed manually
- May drift from Git
- Does not always match the desired state
- Argo CD continuously compares and fixes drift

Git Commit vs Direct kubectl Apply

Git Commit (GitOps)	kubectl apply (Manual)
✓ Stored in Git history	✗ No record in Git
✓ Reviewed by team (pull request)	✗ No review process
✓ Argo CD syncs automatically or manually	✗ One-time manual change
✓ Consistent across environments	✗ Can create configuration drift
✓ Easy to rollback using Git	✗ Harder to track and rollback
✓ Represents the intended state	✗ Only changes the live cluster state

Practical Focus



1. Make a small change to a Kubernetes manifest (e.g. change replica count).
2. Commit and push the change to your Git repository.
3. Observe Argo CD detect and sync the change to the cluster.
4. Compare this with running **kubectl apply** directly and discuss which method is recorded as the intended state and why.

3. Meet Argo CD: The Engineer Watching Your Cluster

VERIQTA

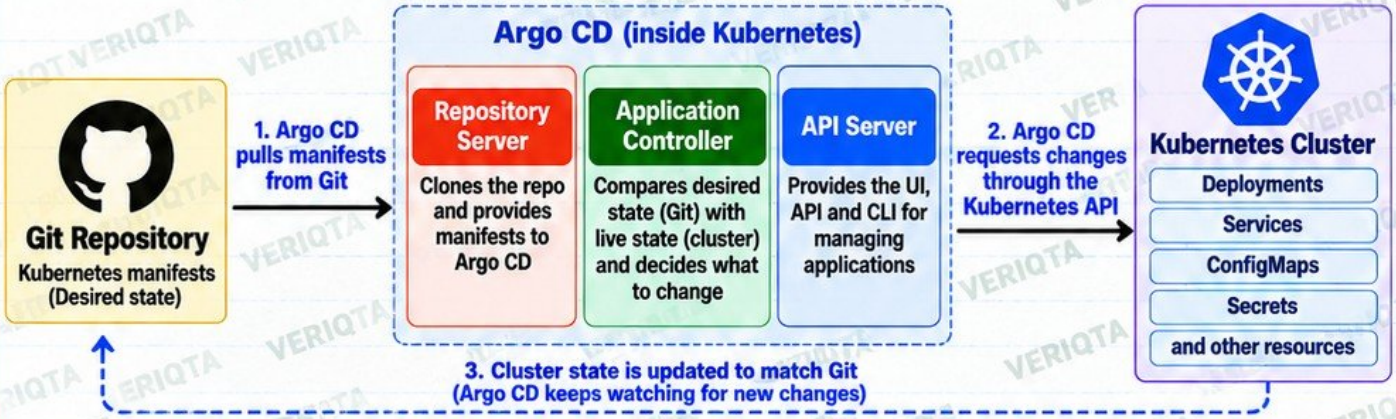
What is Argo CD?

- Argo CD is a Kubernetes-native continuous delivery controller.
- It watches your Git repository for changes and keeps your cluster in the desired state.
- It automatically compares the configuration in Git with the live state in Kubernetes and applies the required changes.

Why Engineers Use Argo CD

- Implements GitOps.
- Reduces manual deployments.
- Keeps environments consistent.
- Provides a clear audit trail.
- Detects and fixes configuration drift.
- Makes deployments repeatable and reliable.
- Works with multiple clusters and environments.

Argo CD Architecture (Simple View)



Component Responsibilities

Application Controller	• Watches applications. • Compares desired vs live state. • Detects drift and triggers sync. • Interfaces with Kubernetes API.
Repository Server	• Clones Git repositories. • Fetches and generates manifests. • Supports Helm, Kustomize and plain YAML.
API Server	• Provides the web UI. • Exposes a REST API and CLI (argocd). • Manages authentication and RBAC. • Stores application and project data.

How Argo CD Works (Flow)

1. Pulls your manifests from Git (repository server).
2. Compares the desired state (Git) with the live state in the cluster.
3. Shows the differences (OutOfSync).
4. When you sync (manual or automatic), it applies the required changes through the Kubernetes API.
5. Continues to watch for new changes in Git.

Key Terms

Desired State	The configuration in your Git repository.
Live State	The actual resources running in Kubernetes.
Sync	The process of making the live state match the desired state.
OutOfSync	Git and cluster state are different.
Healthy	All resources are running and ready.
Degraded	Resources exist but have issues (e.g. failing pods).
Synced	The cluster matches the configuration in Git.

Practical Focus

1. Watch how Argo CD pulls manifests from Git.
2. Observe how it compares with the live cluster state.
3. Trigger a sync and see the changes applied through the Kubernetes API.
4. Understand the role of each component in the process.

4. What Must Exist Before Your First Deployment?

VERIQTA

Key Components You Need to Understand

Kubernetes Cluster
A set of machines that run your containerized applications.

kubeconfig
Tells kubectl how to connect to your cluster and which user/cluster to use.

Namespace
A logical separation for resources (e.g. dev, staging, production).

Deployment
Manages your application Pods and ensures the desired number of replicas.

Service
Exposes your application inside or outside the cluster.

Container Image
A packaged version of your application (e.g. from Docker Hub or a registry).

YAML Manifests
Configuration files that define Kubernetes resources (Deployment, Service, etc).

How They Work Together



Argo CD Prerequisites vs Optional Tools

Required Prerequisites
1. Kubernetes cluster (v1.20 or later)
2. kubectl installed and working
3. kubeconfig configured
4. Namespace for Argo CD (e.g. argocd)
5. Access to a Git repository (e.g. GitHub, GitLab, Bitbucket)
6. Kubernetes permissions (to install Argo CD)

Optional but Useful
1. Helm (alternative installation method)
2. Ingress controller (for web UI access)
3. Load balancer (for production access)
4. Private container registry (for your own images)
5. CI tools (e.g. GitHub Actions) (for building images)
6. Monitoring tools (e.g. Prometheus, Grafana)

Important Concepts (Brief Explanation)

Cluster	A group of machines (nodes) that run Kubernetes.
kubeconfig	A file with cluster connection details, users and contexts.
Namespace	Separates resources (e.g. dev, staging, prod).
Deployment	Ensures the correct number of application Pods are running.
Service	Provides a stable network endpoint for your application.
Container Image	A ready-to-run application package (e.g. nginx, your own app).
YAML Manifest	A text file that defines Kubernetes resources.

Practical Focus

1. Run **kubectl cluster-info** and explain the output.
2. Run **kubectl get nodes** and identify the node status.
3. Run **kubectl config current-context** and confirm the cluster you are connected to.
4. Understand what each command proves and what it does NOT prove.

Verify Your Kubernetes Environment

1 Check cluster information

```
$ kubectl cluster-info
```

What it shows:
- The cluster API server endpoint
- That your kubectl can reach the cluster

What it does NOT show:
- It does not confirm your permissions to create or modify resources.

2 List cluster nodes

```
$ kubectl get nodes
```

What it shows:
- The nodes in your cluster
- Their status (Ready or NotReady)

What it does NOT show:
- It does not confirm that you can deploy applications to a specific namespace.

3 Check current context

```
$ kubectl config
  current-context
```

What it shows:
- Which cluster and user you are using
- Helps you avoid deploying to the wrong environment

What it does NOT show:
- It does not verify your access level or what you are allowed to do.

5. Install Argo CD Inside Kubernetes

VERIQTA

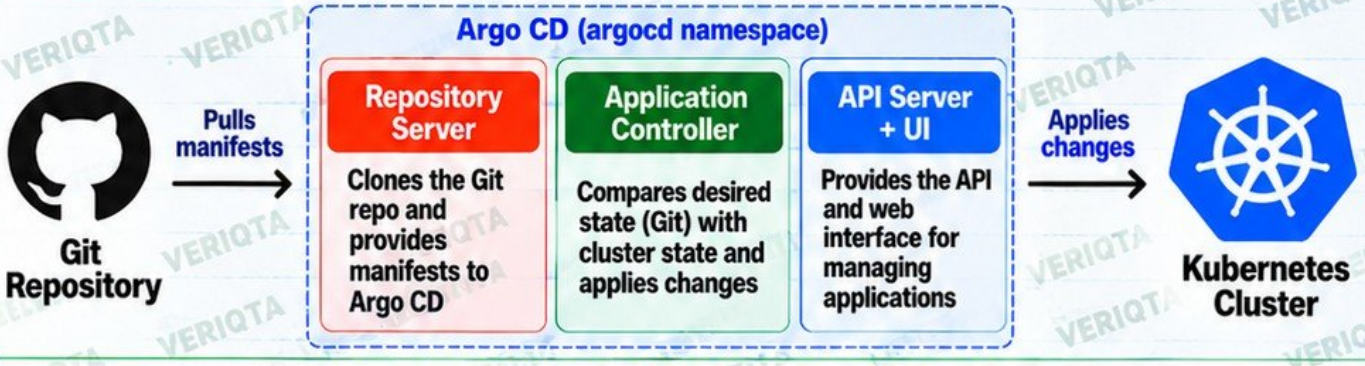
What is Argo CD Installation?

Argo CD is installed inside your Kubernetes cluster as a group of components running in the "argocd" namespace. It uses official installation manifests provided by the Argo CD team.

Why It Runs as Multiple Components

- Each component has a specific responsibility.
- Keeps the system scalable and reliable.
- Separates UI, Git access, reconciliation and APIs.
- Allows independent updates and troubleshooting.
- Follows Kubernetes best practices.

Argo CD Components (Simplified View)



Step 1: Create the argocd Namespace

```
kubectl create namespace argocd
```

✓ This namespace will hold all Argo CD components.

Step 2: Install Argo CD (Use a Specific Version)

```
kubectl apply -n argocd \
  -f https://github.com/argoproj/argo-cd/
 raleacjns/download/v2.10.2/install.yaml
```

✓ Use a specific version (e.g. v2.10.2) instead of "latest".

✓ Pinning the version ensures reproducible environments and avoids unexpected changes.

Step 3: Verify the Installation

```
kubectl get pods -n argocd
```

✓ You should see multiple Pods running.

```
kubectl get svc -n argocd
```

✓ Shows the Argo CD services.

Example Output (Pods)

NAME	READY	STATUS	RESTARTS	AGE
argocd-application-controller-0	1/1	Running	0	1m
argocd-dex-server-0	1/1	Running	0	1m
argocd-notifications-controller-0	1/1	Running	0	1m
argocd-redis-0	1/1	Running	0	1m
argocd-repo-server-0	1/1	Running	0	1m
argocd-server-0	1/1	Running	0	1m

Important Notes

⚠
- Always use a specific version for stable and repeatable environments.
- Installation may take a few minutes for all Pods to be ready.
- All components run in the argocd namespace.
- The installation does not create any applications yet.
- You can access the UI using port-forwarding (next page).

Common Issues

✗
- Some Pods stuck in Pending (check cluster resources).
- ImagePullBackOff (check internet access or registry).
- Installing in the wrong namespace.
- Using "latest" instead of a specific version.
- Not waiting for all Pods to be ready.

6. Access Argo CD Without Exposing It Publicly

VERIQTA

What We Will Do

- Access the Argo CD web interface locally using port forwarding.
- Use the initial administrator credentials.
- Log in and explore the Applications view.
- Change the initial password.
- Understand why exposing the dashboard to the internet is unsafe.

What is the Argo CD Web Interface?

- Argo CD provides a web UI to manage applications, view sync status, and configure Git repositories.
- By default, it is only accessible inside the cluster.
- We will use **kubectl port-forward** to access it locally from your own machine.
- This allows secure access without exposing it to the public internet.

How to Access Argo CD Using Port Forwarding

1. Make sure the Argo CD Pods are running.

```
kubectl get pods -n argocd
```

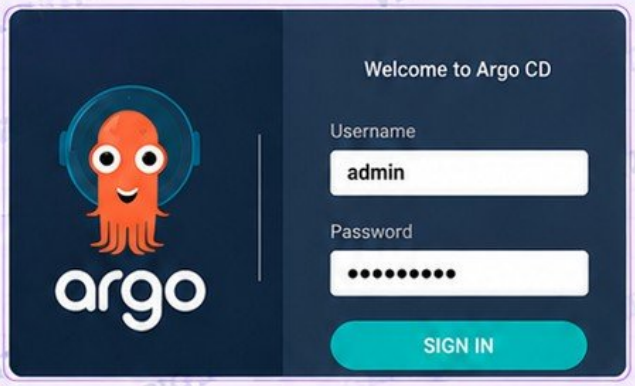
2. Forward the Argo CD server service to your local machine.

```
kubectl port-forward -n argocd svc/argocd-server 8080:443
```

3. Open your browser and go to:

https://localhost:8080

- You should see the Argo CD login page.



Initial Administrator Credentials

- The default username is: **admin**
- The initial password is stored in a Kubernetes Secret.
- Run the following command to get the password:

```
kubectl get secret argocd-initial-admin-secret \
 -n argocd -o jsonpath="{.data.password}" | base64 -d
```

Example Output

```
$ kubectl get secret argocd-initial-admin-secret -n argocd
 -o jsonpath="{.data.password}}" | base64 -d
```

```
dXNlcm5hbWluMTIz
```

- Use the output as your initial password to log in.

Change the Initial Password

1. After logging in, click your profile (top right) → Settings.
2. Choose Change Password.
3. Enter a new strong password and save it.
4. Do not share this password with anyone.

Argo CD Web Interface (Applications View)



Why You Should Not Expose Argo CD Publicly

- It provides admin access to your cluster.
- Anyone who finds the URL can attempt to log in.
- It can be targeted by automated attacks.
- A compromised Argo CD instance can modify or delete your resources.
- Always use port forwarding, VPN, or a secure ingress with authentication and appropriate access controls if external access is required.

Key Takeaways

- Use kubectl port-forward for local, secure access.
- Default username is admin and the password is in a secret.
- Change the initial password immediately.
- Explore the Applications view to manage your deployments.
- Do not expose the Argo CD dashboard to the internet without proper security controls.

7. Build the Git Repository Argo CD Will Watch

VERIQTA

Project Overview

- We will create a Git repository named **veriqta-gitops-lab** to store Kubernetes manifests for our application.
- Argo CD will watch this repository and deploy the manifests to the cluster.
- We will create two main files: **deployment.yaml** and **service.yaml** inside a **k8s** directory.

Repository Structure

```
veriqta-gitops-lab/
└── k8s/
    ├── deployment.yaml    # Application Deployment
    └── service.yaml       # ClusterIP Service
└── README.md              # (optional) project notes
```

File Purpose

deployment.yaml	Defines the application Pods (image, replicas, labels/selectors, ports).
service.yaml	Exposes the application inside the cluster using a ClusterIP service.
k8s/ directory	Holds all Kubernetes manifests for Argo CD.

Step 1: Create the Git Repository

1. Create a new repository on GitHub named **veriqta-gitops-lab** (public or private).
2. Clone the repository to your local machine.

```
git clone https://github.com/your-username/
veriqta-gitops-lab.git
```

3. Create the **k8s** directory and files.

```
mkdir -p veriqta-gitops-lab/k8s
cd veriqta-gitops-lab
```

4. Add the manifests, commit and push.

```
git add .
git commit -m "Add initial Kubernetes manifests"
git push origin main
```

5. Verify the files are on GitHub.

deployment.yaml (Example)

```
apiVersion: apps/v1
kind: Deployment
metadata:
  name: veriqta-demo
  labels:
    app: veriqta-demo
spec:
  replicas: 2
  selector:
    matchLabels:
      app: veriqta-demo
  template:
    metadata:
      labels:
        app: veriqta-demo
    spec:
      containers:
      - name: web
        image: nginx:1.25      # image tag
        ports:
        - containerPort: 80    # container port
```

Key Points

1. **replicas: 2**
   Runs 2 Pods.
2. **selector**
   Matches Pods using label app: veriqta-demo.
3. **image: nginx:1.25**
   Uses a specific image tag for consistency.
4. **containerPort: 80**
   Port the container listens on inside the Pod.

service.yaml (Example)

```
apiVersion: v1
kind: Service
metadata:
  name: veriqta-demo
spec:
  type: ClusterIP
  selector:
    app: veriqta-demo
  ports:
  - port: 80          # Service port
    targetPort: 80    # Container port
```

Key Points

1. **type: ClusterIP**
   Exposes the application only inside the cluster.
2. **selector**
   Matches the Pods using the same label.
3. **port: 80**
   Port the Service listens on.
4. **targetPort: 80**
   Forwards traffic to the container port.

Understanding Important Fields

Field	What It Means
Image tag	Specifies the exact version of the container image (e.g. **nginx:1.25**). Avoid using **latest**.
Replicas	Number of Pod instances to run.
Labels & Selectors	Labels identify Pods, and selectors tell the Deployment and Service which Pods to manage.
Container port	The port the application listens on inside the container.
Service port	The port exposed by the Service inside the cluster.
targetPort	Forwards traffic from the Service to the container port.

Practical Focus



1. Commit a working deployment.yaml and service.yaml using **nginx:1.25**.
2. Explain what each field does (image tag, replicas, selectors, ports).
3. Push the changes to GitHub and confirm the files are in the repository.
4. Ensure the manifests are valid YAML (no syntax errors).

Common Mistakes

- ✗ Using latest tag instead of a specific version.
- ✗ Mismatched labels and selectors.
- ✗ Wrong container port or targetPort.
- ✗ Invalid YAML indentation or syntax.

8. Create Your First Argo CD Application

VERIQTA

What is an Argo CD Application?

- An Application is a custom resource (CR) in Kubernetes that tells Argo CD what to deploy and where.
- It points to a Git repository, a path inside the repo, and a target revision (branch, tag or commit).
- Argo CD reads the manifests and synchronizes them to the specified cluster and namespace.

Key Fields (Explained)

repoURL	URL of your Git repository (e.g. GitHub).
targetRevision	The branch, tag or commit to deploy (e.g. main).
path	Path inside the repository to the manifests (e.g. k8s/).
destination	Which cluster and namespace to deploy to.
project	Argo CD Project to use (default is default).

Application Manifest (veriqta-demo)

```yaml
apiVersion: argoproj.io/v1alpha1
kind: Application
metadata:
  name: veriqta-demo
  namespace: argocd
spec:
  project: default
  source:
    repoURL: https://github.com/your-username/veriqta-gitops-lab.git
    targetRevsion: main
    path: k8s
  destination:
    server: https://kubernetes.default.svc
    namespace: veriqta-demo
  syncPolicy:
    automated: {}  # leave empty for manual sync
    syncOptions:
      - CreateNamespace=true
```

Application name

Git repository URL

Branch, tag or commit

Path to manifests

Cluster API (in-cluster)

Target namespace

Manual sync (no automatic sync)

How to Create the Application

1. Save the manifest to a file (e.g. veriqta-demo.yaml).
2. Apply it to the cluster.

```
kubectl apply -f veriqta-demo.yaml
```

3. Check the application in Argo CD.

```
kubectl get applications -n argocd
```

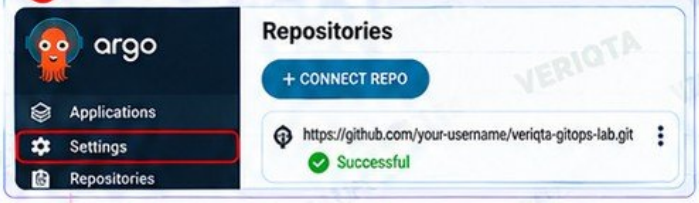
4. Or use the Argo CD CLI (optional).

```
argocd login localhost:8080
argocd app list
```

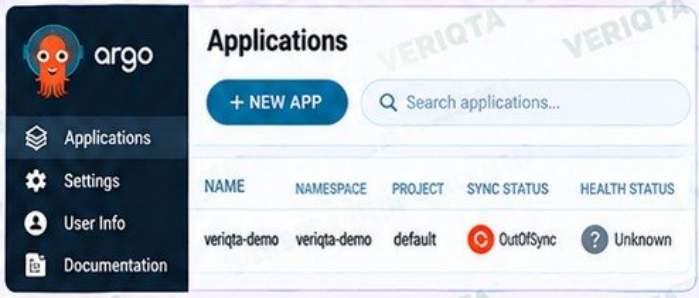
5. The application will be created in an OutOfSync state (manual sync).

Register the Repository (UI Method)

1. Open Argo CD web UI (localhost:8080).
2. Go to Settings → Repositories.
3. Click CONNECT REPO and add your GitHub repo.
4. Test the connection and save.



Initial Application Status (Manual Sync)



Manual Synchronization

1. Open the application in Argo CD UI.
2. Click SYNCHRONIZE.
3. Confirm the sync and wait for it to complete.
4. The status should change to Synced and Healthy (if the manifests are valid).

After Successful Sync



Important Notes

- Use the correct repo URL and path.
- Start with manual sync for safety.
- Create the target namespace or use CreateNamespace=true.
- You can also create the Application from the UI instead of a YAML file.
- If the app stays OutOfSync, check the repo, path and manifests.

9. Your First Sync: From Git to Running Pods

VERIQTA

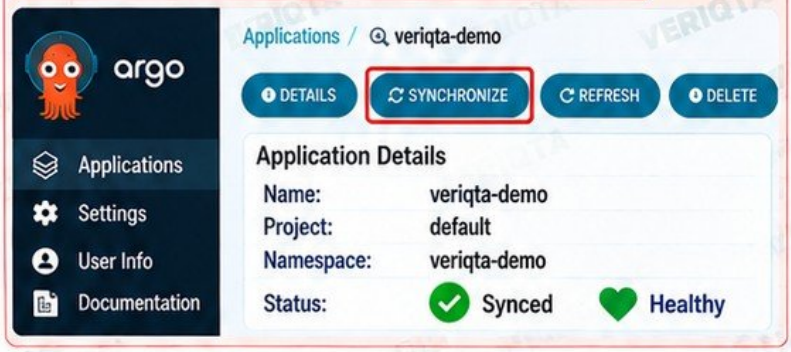
What Happens During the First Sync?

1. Argo CD reads the manifests from your Git repository.
2. It compares the desired state (Git) with the live state (cluster).
3. It creates or updates the resources (Deployment, Service, etc.) in the target namespace.
4. The application status changes from OutOfSync to Synced and becomes Healthy when the Pods are running.

Application Sync Status Explained

OutOfSync	The live state in the cluster does not match the desired state in Git.
Synced	The live state matches the desired state in Git.
Missing	A resource defined in Git does not exist in the cluster.
Healthy	The resources are running and passing health checks (e.g. Pods are ready).

Step 1: Sync the Application (UI Method)



Step 2: Sync the Application (CLI Method)

```
# Trigger a sync
argocd app sync veriqta-demo

# Check application status
argocd app get veriqta-demo

# Watch the sync progress
argocd app wait veriqta-demo --health
```

After a successful sync, the status should be **Synced** and **Healthy**.

Step 3: Verify Resources in the Cluster

Check the Deployment

```
kubectl get deployments -n veriqta-demo
```

NAME	READY	UP-TO-DATE	AVAILABLE	AGE
veriqta-demo	2/2	2	2	2m

Check the Pods

```
kubectl get pods -n veriqta-demo
```

NAME	READY	STATUS	RESTARTS	AGE
veriqta-demo-7c9f6c8d9-abc12	1/1	Running	0	2m
veriqta-demo-7c9f6c8d9-def34	1/1	Running	0	2m

Check the Service

```
kubectl get services -n veriqta-demo
```

NAME	TYPE	CLUSTER-IP	PORT(S)	AGE
veriqta-demo	ClusterIP	10.96.123.45	80/TCP	2m

Step 4: Access the Application (Local Port Forward)

```
# Forward the service to your local machine
kubectl port-forward svc/veriqta-demo 8080:80 \
   -n veriqta-demo
```

Open your browser and go to:

http://localhost:8080

You should see your application running.

Example: Application Status Before and After Sync

Before Sync

veriqta-demo

OutOfSync ? Unknown

Namespace:	veriqta-demo
Project:	vefault
Source:	main
Path:	k8s

After Sync

veriqta-demo

Synced Healthy

Namespace:	veriqta-demo
Project:	default
Source:	main
Path:	k8s

Key Takeaways

- Sync applies the manifests from Git to your cluster.
- Argo CD creates the Deployment and Service automatically.
- OutOfSync means the live state is different from Git.
- Synced means the live state matches Git.
- Healthy means the application is running correctly.
- You can verify the resources with kubectl commands and access the app using port-forwarding.

Common Issues

- Application stays OutOfSync (check manifests).
- Pods not running (check image, labels, selectors).
- Service not accessible (check service type and ports).
- Wrong namespace in the destination.
- Image pull errors (check image name and tag).
- Application shows Missing (resource not created).

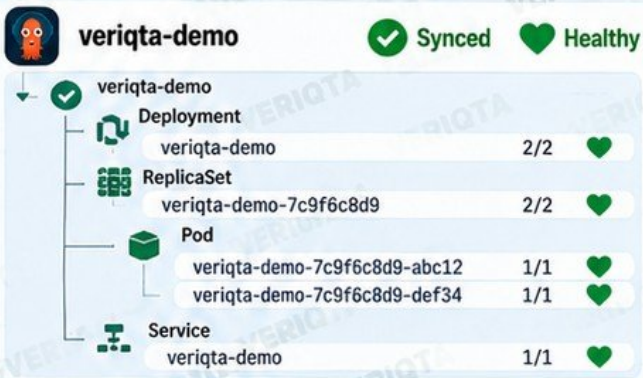
10. Prove the Deployment Actually Works

VERIQTA

Why Verification is Important?

- A successful Argo CD sync only means the resources were created and match the Git definition.
- It does not guarantee that your application is actually running or accessible.
- You must check Pod readiness, Service endpoints, and the HTTP response to confirm it works.
- This ensures the deployed application is usable by real users.

Resource Tree in Argo CD (UI)



Step 1: Check Deployment Readiness

Verify the Deployment has the desired number of ready replicas.

```
kubectl get deployment veriqta-demo -n veriqta-demo
```

```
NAME           READY   UP-TO-DATE   AVAILABLE   AGE
veriqta-demo   2/2     2            2           5m
```

- READY and AVAILABLE should match the replica count in your manifest.

Step 2: Check Pod Status

Ensure all Pods are Running and Ready.

```
kubectl get pods -n veriqta-demo
```

```
NAME                            READY   STATUS    RESTARTS   AGE
veriqta-demo-7c9f6c8d9-abc12    1/1     Running   0          5m
veriqta-demo-7c9f6c8d9-def34    1/1     Running   0          5m
```

- Pods should be in Running state.
- READY should be 1/1.

Step 3: Check Service and Endpoints

Verify the Service exists and has healthy endpoints.

```
kubectl get svc -n veriqta-demo
```

```
NAME           TYPE        CLUSTER-IP     PORT(S)   AGE
veriqta-demo   ClusterIP   10.96.123.45   80/TCP    5m
```

```
kubectl get endpoints -n veriqta-demo
```

```
NAME           ENDPOINTS                          AGE
veriqta-demo   10.244.1.12:80,10.244.2.15:80      5m
```

- Service should have a ClusterIP and correct port.
- Endpoints should show the Pod IPs and port 80.

Step 4: Test the Application (HTTP Request)

Access the service using port-forwarding and make an HTTP request.

```
kubectl port-forward svc/veriqta-demo 8080:80 \
  -n veriqta-demo
```

- In another terminal, test the application:

```
curl -I http://localhost:8080
```

or

```
curl http://localhost:8080
```

```
HTTP/1.1 200 OK
Server: nginx/1.25
Content-Type: text/html
Content-Length: 615
...
```

- You should receive HTTP 200 OK and a valid response from your application.

What to Look For

Component	What to Verify	Expected Result
Application (Argo CD)	Synced and Healthy	Synced + Healthy
Deployment	Ready replicas	READY matches replicas
Pods	Running and Ready	1/1 Running
Service	ClusterIP and correct port	ClusterIP, Port 80
Endpoints	Pod IPs listed	Shows Pod IPs and port 80
HTTP Response	Application returns response	HTTP 200 OK

Key Takeaways

- A successful sync does not mean the application is usable.
- Always check Deployment, Pods, Service, and Endpoints.
- Use port-forwarding for local access during testing.
- A working HTTP response proves the application is running.
- Keep evidence (screenshots or command output) of a working release.

Common Issues

- Pods are Running but Service has no endpoints.
- Service exists but wrong port or targetPort.
- Application returns 404 or 500 (check container logs).
- Using the wrong namespace or local port.
- Firewall or network policy blocking access.

11. Change Git and Watch Kubernetes Respond

VERIQTA

Why Change Through Git?

- The desired state is stored in Git and version controlled.
- Changing Git provides a clear history, review process, and the ability to roll back.
- Argo CD detects the change and shows the application as OutOfSync until you synchronize.
- This is different from manually editing a llive Kubernetes resource with kubectl, which is not recorded as the intended state and will be overwritten by Argo CD.

What We Will Do

1. Update the Deployment replica count in Git.
2. Commit and push the change.
3. Refresh Argo CD and check the application status.
4. Compare the difference before synchronizing.
5. Manually synchronize and verify Kubernetes creates the new Pods.

Step 1: Update the Deployment in Git

Edit deployment.yaml and change replicas from 2 to 3.

Before (replicas: 2)

```yaml
apiVersion: apps/v1
kind: Deployment
metadata:
  name: veriqta-demo
spec:
  replicas: 2
  selector:
    matchLabels:
      app: veriqta-demo
```

After (replicas: 3)

```yaml
apiVersion: apps/v1
kind: Deployment
metadata:
  name: veriqta-demo
spec:
  replicas: 3
  selector:
    matchLabels:
      app: veriqta-demo
```

Step 2: Commit and Push the Change

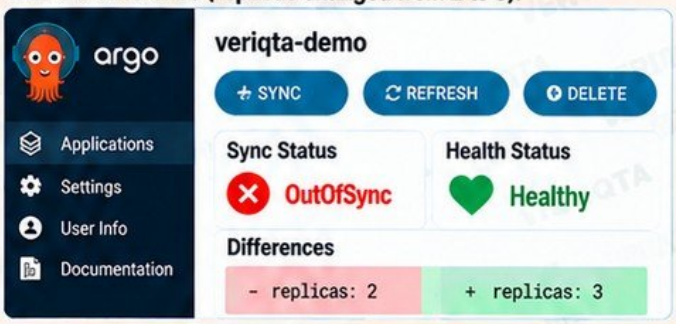
Commit the change with a clear message and push to GitHub.

```
git add k8s/deployment.yaml
git commit -m "Increase replicas to 3"
git push origin main
```

The change is now recorded in Git and Argo CD will detect it automatically.

Step 3: Check the Status in Argo CD (Before Sync)

Refresh the application in Argo CD. It should show **OutOfSync** with the difference (replicas changed from 2 to 3).



Step 4: Inspect the Difference

Argo CD shows what will change before synchronization.

Differences

```
spec:
- replicas: 2
+ replicas: 3
  selector:
    matchLabels:
      app: veriqta-demo
```

What This Means?

Argo CD has detected a change in the desired state (Git) that does not match the live state (cluster). The application is OutOfSync until you synchronize it.

Step 5: Manually Synchronize

Click SYNCHRONIZE in Argo CD UI or use the CLI.

```
# CLI method
argocd app sync veriqta-demo
```



Argo CD will apply the changes to the cluster.

Step 6: Verify in Kubernetes

Check that the new number of replicas and Pods are running.

```
kubectl get deployment veriqta-demo -n veriqta-demo

NAME           READY   UP-TO-DATE   AVAILABLE   AGE
veriqta-demo   3/3     3            3           5m
```

```
kubectl get pods -n veriqta-demo

NAME                            READY   STATUS    RESTARTS   AGE
veriqta-demo-7c9f6c8d9-abc12    1/1     Running   0          1m
veriqta-demo-7c9f6c8d9-def34    1/1     Running   0          1m
veriqta-demo-7c9f6c8d9-ghi56    1/1     Running   0          1m
```

Key Takeaways

- Changing Git is the correct and safe way to update your application.
- Argo CD detects the change and shows OutOfSync.
- You can review the differences before synchronizing.
- After synchronization, Kubernetes creates the new resources.
- The change is recorded in Git and you can roll back if needed.

Common Mistakes

- Editing resources directly with kubectl instead of Git.
- Forgetting to commit and push the change.
- Expecting the cluster to change without synchronizing (in manual sync mode).
- Using the wrong branch, path, or file.
- Not verifying the new Pods and application after sync.

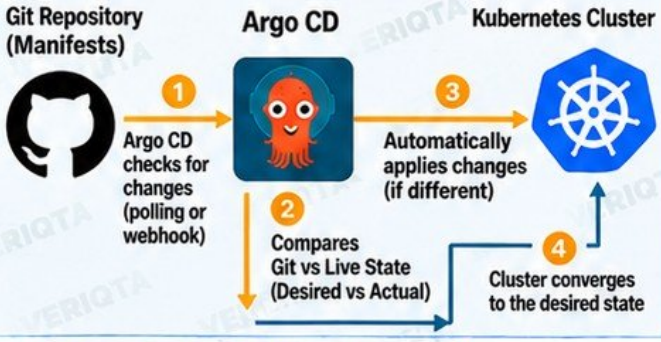
12. Automatic Sync: Deploy Without Clicking Sync

VERIQTA

What is Automatic Synchronization?

- Automatic sync tells Argo CD to automatically apply changes from Git to your cluster.
- Argo CD continuously checks the repository using polling or supported notifications (e.g. webhooks).
- When it detects a change, it compares the desired state with the live state and applies the changes.
- It is not always instant. By default, Argo CD polls the repository at a set interval (e.g. every 3 minutes) unless you configure webhooks.

Reconciliation Flow (How It Works)



Step 1: Enable Automatic Sync

Add the automated section to your Application manifest.

```yaml
apiVersion: argoproj.io/v1alpha1
kind: Application
metadata:
  name: veriqta-demo
  namespace: argocd
spec:
  project: default
  source:
    repoURL: https://github.com/your-username/veriqta-gitops-lab.git
    targetRevision: main
    path: k8s
  destination:
    server: https://kubernetes.default.svc
    namespace: veriqta-demo
  syncPolicy:
    automated:
      prune: true      # remove resources not in Git
      selfHeal: true   # fix manual changes in the cluster
    syncOptions:
      - CreateNamespace=true
```

Key Fields

1. **automated:** enables automatic synchronization.
2. **prune:** deletes resources removed from Git.
3. **selfHeal:** reverts manual changes in the cluster.

Step 2: Apply the Change

1. Increase replicas from 3 to 4 in deployment.yaml.
2. Commit and push the change to GitHub.
3. Wait for Argo CD to detect and apply the change automatically (no manual sync).

```
# Edit the file and change replicas to 4
git add k8s/deployment.yaml
git commit -m "Increase replicas to 4 (auto sync)"
git push origin main
```

Step 3: Observe Automatic Synchronization

- Open the Argo CD UI and watch the status change.
- You will see OutOfSync briefly, then Synced and Healthy after Argo CD applies the changes.
- No manual sync is required.



Step 4: Verify in Kubernetes

Check that the new number of replicas and Pods are running without a manual sync.

```
kubectl get deployment veriqta-demo -n veriqta-demo

NAME           READY   UP-TO-DATE   AVAILABLE   AGE
veriqta-demo   4/4     4            4           2m
```

```
kubectl get pods -n veriqta-demo

NAME                            READY   STATUS    RESTARTS   AGE
veriqta-demo-5d7f8c7b9-abc12    1/1     Running   0          2m
veriqta-demo-5d7f8c7b9-def34    1/1     Running   0          2m
veriqta-demo-5d7f8c7b9-ghi56    1/1     Running   0          2m
veriqta-demo-5d7f8c7b9-jk178    1/1     Running   0          2m
```

Key Takeaways

- Automatic sync deploys changes from Git without clicking Sync.
- Argo CD detects changes through polling or supported notifications.
- There can be a small delay depending on the polling interval.
- Use prune and selfHeal to keep the cluster consistent with Git.
- Always verify the result in Kubernetes.

Common Mistakes

- Expecting instant deployment (polling has an interval).
- Not enabling prune or selfHeal.
- Using the wrong branch, path, or file.
- Forgetting to commit and push the change.
- Not verifying the resources in the cluster.

13. Configuration Drift: When Someone Changes the Cluster

VERIQTA

What is Configuration Drift?

- Configuration drift happens when the live Kubernetes resources are different from what is defined in Git.
- It can happen when someone manually changes a resource using kubectl, the Kubernetes dashboard, or another tool.
- Argo CD detects the drift and shows the application as OutOfSync.
- Self-healing can automatically revert the changes and restore the Git-declared state.

Drift vs Self-Healing (Key Difference)

Drift Detection	Self-Healing
Detects that the live state is different from Git.	Automatically reverts the live state to match Git.
Shows OutOfSync in Argo CD.	Only works when automated sync and selfHeal are enabled.
Does not fix the change by itself.	Returns the resource to the Git-declared configuration.
Lets you see what changed before taking action.	Helps prevent unauthorized or accidental changes.

Step 1: Verify the Current State (From Git)

The Deployment is set to 3 replicas in Git and is running correctly.

```
kubectl get deployment veriqta-demo -n veriqta-demo

NAME           READY   UP-TO-DATE   AVAILABLE   AGE
veriqta-demo   3/3     3            3           5m
```

This matches the Git configuration (replicas: 3).

Step 2: Manually Change the Cluster (Create Drift)

Someone manually scales the Deployment to 5 replicas using kubectl (not through Git).

```
kubectl scale deployment veriqta-demo -n veriqta-demo \
  --replicas=5
```

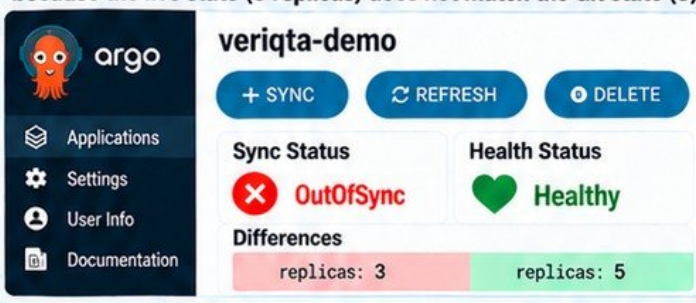
```
kubectl get deployment veriqta-demo -n veriqta-demo

NAME           READY   UP-TO-DATE   AVAILABLE   AGE
veriqta-demo   5/5     5            5           6m
```

The live cluster now has 5 replicas (different from Git).

Step 3: Observe Drift in Argo CD

Refresh the application in Argo CD. It will show OutOfSync because the live state (5 replicas) does not match the Git state (3).



Step 4: Enable Self-Healing

Update the application to enable automated sync and self-healing.

```
spec:
  syncPolicy:
    automated:
      prune: true
      selfHeal: true
    syncOptions:
    - CreateNamespace=true
```

Key Fields

1. **automated:** enables automatic synchronization.
2. **selfHeal:** reverts manual changes in the cluster.
3. **prune:** removes resources not in Git (optional).

Step 5: Watch Argo CD Restore the Correct State

After enabling self-healing, Argo CD will automatically detect the drift and restore the Git-declared replica count (3).

```
kubectl get deployment veriqta-demo -n veriqta-demo

NAME           READY   UP-TO-DATE   AVAILABLE   AGE
veriqta-demo   3/3     3            3           8m
```

Argo CD has automatically reverted the manual change and restored the cluster to the Git state.

What to Look For

Check	Expected Result
Argo CD Sync Status	OutOfSync after manual change, then Synced after self-healing.
Live replicas (kubectl)	Shows 5 after manual scale, then returns to 3 after self-healing.
Application health	Stays Healthy if Pods are running.
Argo CD events/logs	Shows the automatic synchronization and restored change.

Key Takeaways

- Configuration drift happens when the live state is different from Git.
- Argo CD detects drift and shows OutOfSync.
- Self-healing automatically restores the Git-declared configuration.
- Always use Git to make changes so the desired state is consistent and auditable.

Common Mistakes

- Manually changing resources in the cluster.
- Expecting Argo CD to fix changes without self-healing.
- Not enabling automated sync.
- Not checking the application status after the change.
- Changing the wrong namespace or resource.

14. Pruning: What Happens When Git Removes a Resource?

VERIQTA

What is Pruning?

- Pruning removes Kubernetes resources from the cluster when they are no longer defined in the Git repository.
- It keeps the cluster in sync with your Git repository by deleting resources that were removed from Git.
- This prevents orphaned resources and configuration drift.
- Pruning can be done manually or automatically.
- Be careful in production, because pruning will permanently delete resources from the cluster.

Manual Prune vs Automated Prune

Manual Prune

- You review the resources that will be deleted.
- You click Prune to remove them.
- Safer because you can see what will be deleted.
- Good for testing and production environments.

Automated Prune

- Automatically deletes resources removed from Git.
- Works with automated sync.
- No manual approval required.
- Helpful for fully Git-driven environments.
- Can be dangerous if someone accidentally removes important resources from Git.

Step 1: Add a ConfigMap to Git

Create a simple ConfigMap and commit it to your repository.

```yaml
# k8s/configmap.yaml
apiVersion: v1
kind: ConfigMap
metadata:
  name: veriqta-message
  namespace: veriqta-demo
data:
  message: "Hello from VERIQTA GitOps Lab!"
```

Step 2: Synchronize the Application

Sync the application in Argo CD. The ConfigMap will be created in the cluster.




```
kubectl get configmap -n veriqta-demo
NAME                DATA      AGE
veriqta-message     1         2m
```

Step 3: Remove the ConfigMap from Git

Delete configmap.yaml from your repository and commit the change.

```
git rm k8s/configmap.yaml
git commit -m "Remove ConfigMap"
git push origin main
```

The ConfigMap no longer exists in the Git repository.

- veriqta-gitops-lab
  - k8s/
    - deployment.yaml
    - service.yaml
    - ✗ configmap.yaml (removed)

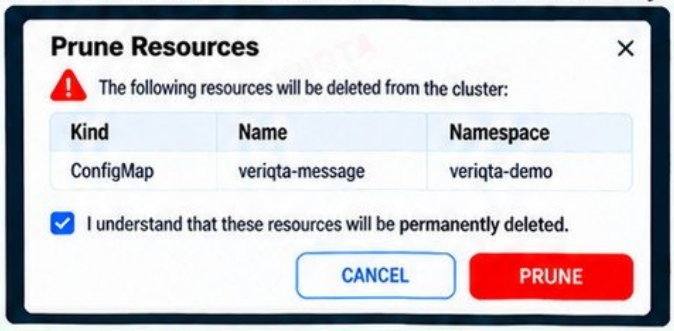
Step 4: Check the Difference in Argo CD

Refresh the application. Argo CD will show the ConfigMap as Missing and the application will be OutOfSync.



Step 5: Preview the Deletion (Manual Prune)

Click PRUNE to see what will be deleted. Review the resources carefully.



Step 6: Prune and Verify

After confirming, click PRUNE. Argo CD will delete the ConfigMap from the cluster and the application will return to Synced.

```
kubectl get configmap -n veriqta-demo
No resources found in veriqta-demo namespace.
```

The ConfigMap has been removed from the cluster and the application is Synced again.

Enable Automated Prune (Optional)

To enable automated pruning, update the Application manifest.

```yaml
spec:
  syncPolicy:
    automated:
      prune: true
      selfHeal: true
```

Key Takeaways

- Pruning deletes resources removed from Git.
- Manual prune lets you review before deletion.
- Automated prune deletes automatically (use with care).
- Always verify the resources to be deleted.
- Pruning keeps the cluster clean and aligned with Git.

Common Mistakes

- ✗ Accidentally removing important resources from Git.
- ✗ Enabling automated prune in production without proper testing.
- ✗ Not reviewing the deletion preview.
- ✗ Forgetting that pruning permanently deletes resources from the cluster.
- ✗ Removing the wrong file or directory from the repository.

15. Deploy a New Version and Verify the Rollout

VERIQTA

Key Concepts

- Image tags identify a specific version of a container image.
- A Deployment creates a new ReplicaSet when the image changes.
- Old Pods are replaced with new Pods using a rolling update.
- Changing a container image does not guarantee a successful release.
- Always verify Pod readiness, application health, and the HTTP response.
- Use immutable image tags (e.g. v1.0.0) instead of latest for predictable deployments.
- Check image provenance to ensure you are using trusted image.

How a Deployment Rollout Works

1. **Update image in Git**
   Change the image tag in deployment.yaml
2. **Argo CD detects change**
   Syncs the new manifest to the cluster
3. **New ReplicaSet created**
   A new ReplicaSet with the new image is created
4. **Rolling update happens**
   New Pods become ready, old Pods are terminated
5. **Verify the application**
   Check Pods, ReplicaSets and test the HTTP response

Example Image Tags

nginx:1.25.3	(specific version)
nginx:1.26.0	(new version)
nginx:latest	(not recommended)

Why Immutable Tags?

- Always deploy a known version.
- Avoid unexpected changes.
- Easier to rollback if needed.
- Improves security and traceability.

Important Note

- Changing the image does not guarantee the application will work.
- The container may start but fail health checks.
- Always verify the application after rollout.
- Use trusted images from official sources.

Step 1: Update the Image in Git

Edit deployment.yaml and change the image tag to a new version.

```yaml
apiVersion: apps/v1
kind: Deployment
metadata:
  name: veriqta-demo
spec:
  replicas: 3
  selector:
    matchLabels:
      app: veriqta-demo
  template:
    metadata:
      labels:
        app: veriqta-demo
    spec:
      containers:
      - name: app
        image: nginx:1.26.0  # new version
        ports:
        - containerPort: 80
```

Git Commands

```
git add k8s/deployment.yaml
git commit -m "Update
   image to 1.26.0"
git push origin main
```

What Changed?

- The image tag was updated from nginx:1.25.3 to nginx:1.26.0.
- This will trigger a new ReplicaSet and rolling update.
- Argo CD will detect the change and deploy it.

Step 2: Observe the Rollout in Kubernetes

Check the new ReplicaSet, Pods and their status.

```
kubectl get rs -n veriqta-demo
NAME                      DESIRED   CURRENT   READY   AGE
veriqta-demo-7bdc9c6d4    0         0         0       5m
veriqta-demo-5f9d8b7c6    3         3         3       30s
```

New ReplicaSet with new image

```
kubectl get pods -n veriqta-demo
NAME                              READY   STATUS    RESTARTS   AGE
veriqta-demo-5f9d8b7c6-abc12      1/1     Running   0          25s
veriqta-demo-5f9d8b7c6-def34      1/1     Running   0          25s
veriqta-demo-5f9d8b7c6-ghi56      1/1     Running   0          25s
```

```
kubectl rollout status deployment/veriqta-demo -n veriqta-demo
deployment "veriqta-demo" successfully rolled out
```

Step 3: Verify the Application (HTTP Response)

Access the application using port-forwarding and check the response.

```
kubectl port-forward svc/veriqta-demo 8080:80 -n veriqta-demo
```

In another terminal, test the application:

```
curl -I http://localhost:8080
```

```
HTTP/1.1 200 OK
Server: nginx/1.26.0
Content-Type: text/html
Content-Length: 615
...
```

- You should receive HTTP 200 OK.
- The response should show the new version (nginx/1.26.0).
- This confirms the new release is running and accessible.

Step 4: Check Application Readiness

The new Pods must pass readiness checks before they receive traffic.

```
kubectl describe pod veriqta-demo-5f9d8b7c6-abc12
Readiness:      http-get http://:80/   delay=0s timeout=1s
Liveness:       http-get http://:80/   delay=0s timeout=1s
```

- Readiness probes ensure the application is ready to serve traffic.
- If the readiness check fails, the Pod will not receive traffic.
- Always verify that all new Pods are in Ready state.

Step 5: Verify Image Provenance

- Use official images or trusted registries (e.g. Docker Hub, ECR, GCR).
- Check the image tag and source.
- Avoid using unverified or unknown images.
- Consider using image signing and scanning for production.

Key Takeaways

- Updating the image tag triggers a rolling update with a new ReplicaSet.
- A successful rollout does not guarantee the application works, always verify.
- Use immutable tags for predictable deployments.
- Verify Pod readiness and the HTTP response.
- Use trusted images and check image provenance.

16. OutOfSync, Degraded, and Missing: Read the Evidence

VERIQTA

Key Concepts

- Synchronization status tells you if the live cluster matches what is in Git.
- Resource health tells you if the deployed workload is actually working.
- Operation status tells you the result of the last sync operation.
- Synced can coexist with an unhealthy workload (e.g. Pods failing) because the cluster state still matches Git.
- OutOfSync means the live state is different from Git.
- Degraded means a resource is not healthy (e.g. failing Pods, crash loops, or misconfigurations).
- Missing means a resource defined in Git is not found in the cluster.

Argo CD Application Status Layers

1. Synchronization Status
Compares Git desired state vs live cluster.
Examples: Synced, OutOfSync.

↓

2. Resource Health
Checks if the deployed resources are working.
Examples: Healthy, Degraded, Missing, Progressing, Unknown.

↓

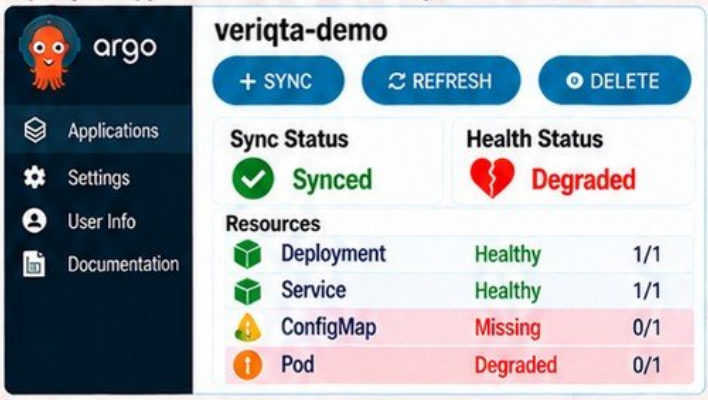
3. Operation Status
Shows the result of the last sync action.
Examples: Succeeded, Failed, Running.

Status Meanings

Synced	Live state matches Git.
OutOfSync	Live state is different from Git.
Healthy	Resource is working as expected.
Degraded	Resource has issues (e.g. failing Pods).
Missing	Resource defined in Git is not found in the cluster.
Progressing	Resource is still being created or updated.
Unknown	Health status cannot be determined.

Step 1: Check Application Status in Argo CD UI

Open your application and look at both sync and health status.



Step 2: Get Detailed Status with Argo CD CLI

Use argocd app get to see sync, health, and operation details.

```
argocd app get veriqta-demo
Name:           veriqta-demo
Project:        default
Server:         https://kubernetes.default.svc
Namespace:      veriqta-demo
URL:            https://argocd.example.com/applications/veriqta-demo
Sync Policy:    Automated (prune=false, selfHeal=false)
Sync Status:    Synced
Health Status:  Degraded
Last Sync:      2026-10-23 10:15:04 (Succeeded)
Revision:       abc1234
Resources:
   Deployment/veriqta-demo          Healthy     1/1
   Service/veriqta-demo             Healthy     1/1
   ConfigMap/veriqta-message        Missing     0/1
   Pod/veriqta-demo-7f9d8           Degraded    0/1
```

Step 3: Investigate with kubectl

Check the actual Kubernetes resources and their status.

```
kubectl get all -n veriqta-demo
NAME                              READY   STATUS             RESTARTS   AGE
pod/veriqta-demo-7f9d8            0/1     CrashLoopBackOff   5          2m
deployment.apps/veriqta-demo      1/1                        1          2m
service/veriqta-demo              1/1                        1          2m
```

```
kubectl describe pod veriqta-demo-7f9d8 -n veriqta-demo
  State:          Waiting
   Reason:        CrashLoopBackOff
   Last State:    Terminated
   Message:       Error: failed to start application
```

```
kubectl get events -n veriqta-demo --sort-by='.lastTimestamp'
LAST SEEN   TYPE      REASON    OBJECT                  MESSAGE
2m          Warning   BackOff   pod/veriqta-demo-7f9d8  Back-off restarting container
2m          Normal    Pulling   pod/veriqta-demo-7f9d8  Pulling image "nginx:bad"
```

Step 4: Identify the Failing Layer

Use the evidence to determine where the problem is.

Evidence	Likely Layer	Example Cause
OutOfSync	Git vs Cluster	Manual change in cluster or pending Git update.
Synced + Degraded	Application / Runtime	Pods failing, bad image, failed readiness check.
Missing	Resource Presence	Resource defined in Git not found in cluster.
Sync Failed	Operation	Invalid manifest, RBAC issue, or cluster error.
Healthy but not working	Application Logic	Application running but returning errors (e.g. 500).

Key Takeaways

- Always look at both Sync Status and Health Status.
- Synced does not mean the application is working.
- Use argocd app get, kubectl describe, and events to find the real issue.
- Check logs and application behavior to confirm the root cause.
- Fix the issue in Git where possible and let Argo CD reconcile the change.

Common Mistakes

- Assuming Synced means the application is healthy.
- Only checking the Argo CD UI and not using kubectl.
- Ignoring Kubernetes events and Pod logs.
- Using the wrong image tag or invalid configuration.
- Not understanding the difference between sync status, resource health, and operation status.

17. Investigate a Failed Deployment

VERIQTA

What We Will Do

- Deploy a version with a nonexistent image tag.
- Observe the failure in Argo CD and in Kubernetes.
- Understand ImagePullBackOff and image retrieval.
- Identify whether the issue is an Argo CD configuration problem or a Kubernetes runtime problem.
- Fix the image tag and verify the application recovers.

What is ImagePullBackOff?

- It means Kubernetes cannot pull the container image from the registry.
- Usually caused by a wrong image tag, nonexistent image, or registry access issues.
- Kubernetes keeps retrying to pull the image with an increasing delay (back-off).
- The Pod stays in a non-running state until the image can be pulled successfully.

Common Causes

1. Nonexistent or incorrect image tag.
2. Private registry without proper authentication.
3. Network or DNS issues.
4. Image removed from registry.
5. Rate limiting from public registries (e.g. Docker Hub).
6. Wrong image name or repository path.

Step 1: Deploy an Invalid Image Tag

Edit deployment.yaml and set a nonexistent image tag.

```yaml
apiVersion: apps/v1
kind: Deployment
metadata:
  name: veriqta-demo
spec:
  replicas: 3
  template:
    spec:
      containers:
      - name: app
        image: nginx:does-not-exist   # invalid tag
        ports:
        - containerPort: 80
```

Step 2: Commit and Sync

Commit the change to Git and sync the application in Argo CD.

```
git add k8s/deployment.yaml
git commit -m "Use invalid
            image tag"
git push origin main
```

Expected Result in Argo CD



veriqta-demo

Sync Status	Health Status
Synced	Degraded

Step 3: Check Pod Status

The Pods will not start and show ImagePullBackOff.

```
kubectl get pods -n veriqta-demo
```

NAME	READY	STATUS	RESTARTS	AGE
veriqta-demo-7f9d8	0/1	ImagePullBackOff	3	2m
veriqta-demo-9c7b6	0/1	ImagePullBackOff	3	2m

Understanding the Status

- READY 0/1: container is not running.
- STATUS ImagePullBackOff: failed to pull image.
- RESTARTS: keeps retrying.
- AGE: time since the Pod was created.

Step 4: Investigate the Failure

Use describe and events to find the root cause.

```
kubectl describe pod veriqta-demo-7f9d8 -n veriqta-demo
Name:           veriqta-demo-7f9d8
Namespace:      veriqta-demo
Status:         Pending
Container Statuses:
  app:
    State:       Waiting
      Reason:    ImagePullBackOff
    Last State:  Terminated
      Reason:    ErrImagePull
Events:
Type     Reason   Age   From      Message
----     ------   ---   ----      -------
Normal   Pulling  2m    kubelet   Pulling image "nginx:does-not-exist"
Warning  Failed   2m    kubelet   Failed to pull image "nginx:does-not-exist":
                                  rpc error: code = NotFound
Warning  Failed   2m    kubelet   Error: ErrImagePull
Normal   BackOff  1m    kubelet   Back-off pulling image "nginx:does-not-exist"
```

Step 5: Verify the Root Cause

Check if the image exists in the registry.

```
docker pull nginx:does-not-exist
Error response from daemon: manifest for nginx:does-not-exist
not found: manifest unknown: manifest unknown
```

What This Tells Us

- The image tag does not exist in the registry.
- This is a Kubernetes runtime problem (image retrieval), not an Argo CD configuration problem.
- Argo CD applied the manifest correctly, but the cluster cannot run the container.

Step 6: Fix the Image Tag

Update the deployment.yaml with a valid image tag.

```yaml
image: nginx:1.26.0   # valid tag
```

Commands

```
git add k8s/deployment.yaml
git commit -m "Use valid image tag"
git push origin main
```

Step 7: Verify the Application Recovers

Check that the new Pods are running and the application is healthy.

```
kubectl get pods -n veriqta-demo
NAME                  READY   STATUS    RESTARTS   AGE
veriqta-demo-6c5f7    1/1     Running   0          1m
```

Key Takeaways

- ImagePullBackOff means Kubernetes cannot pull the container image.
- Check Pod events to find the exact error message.
- A failed deployment can be a runtime issue, even when Argo CD shows Synced.
- Always verify the image tag and registry access.
- Fix the root cause and confirm the application recovers.

Common Mistakes

- Using a nonexistent or incorrect image tag.
- Assuming Argo CD is the problem when it is a runtime issue.
- Not checking Pod events and logs.
- Using a private registry without configuration.
- Ignoring image pull errors and only checking Argo CD status.

18. Recover Safely Using Git History

VERIQTA

Key Concepts

- Git history keeps a record of all changes to your manifests.
- You can recover a known-good state by using Git revert, redeployment, or Argo CD's history/rollback features.
- Git revert creates a new commit that undoes the bad change.
- Redeployment re-applies a previous version of the manifest.
- Argo CD history/rollback lets you return to a previous application state from the Ubr CLI.
- Automated sync can affect rollback by immediately reconciling changes in Git.

Rollback Options

Git Revert

- Creates a new commit that undoes the bad change.
- Safe and auditable.
- Triggers Argo CD sync.

Redeployment

- Reapply a previous manifest version.
- Useful when you need a quick fix.
- Works well with versioned manifests.

Argo CD History / Rollback

- Roll back to a previous app state.
- Available from UI or CLI.
- Keeps a history of syncs.

Why Automated Sync Matters

- With automated sync ON, Argo CD will immediately reconcile changes in Git.
- This can override manual rollback attempts.
- For controlled recovery, consider pausing auto-sync or using ta specific rollback method.

Important

Always understand your sync policy before performing a rollback in production.

Step 1: Revert the Bad Commit

Revert the commit that introduced the bad image tag.

```
git log --oneline
# Find the bad commit hash (e.g. a1b2c3d)
git revert a1b2c3d
# Resolve conflicts if any, then:
git push origin main
```

This creates a new commit that restores the previous, working image reference.

Step 2: Let Argo CD Reconcile

Argo CD will detect the Git change and synchronize the application (if auto-sync is on).

```
#  Check the application status
argocd  app get veriqta-demo

# Or watch for sync
argocd app watch veriqta-demo
```

Look for Sync Status: Synced and Health Status: Healthy.

Step 3: Verify the Rollout

Check the new ReplicaSet and confirm the pods are running with the correct image.

```
kubectl get rs -n veriqta-demo
kubectl get pods -n veriqta-demo
kubectl describe pod <pod-name>
kubectl get events -n veriqta-demo
```

Ensure the new image is running and there are no errors.

Step 4: Confirm the Application Works

Test the application with an HTTP request.

```
curl -I http://localhost:8080
# or
curl -I  http://<node-ip>:<node-port>
# or
curl -I <app-domain>
```

You should see a 200 OK response and the application should work in the browser.

Example: Git History After Revert

```
git log --oneline
c9f8e7a (HEAD -> main) Revert "Update image to ..."
a1b2c3d Bad image tag (failed deployment)
7e6d5c4 Initial working version
```

The revert commit is now the latest change and will be synced by Argo CD.

What to Look For

Check	Expected Result
Git history	Shows a new revert commit at the top.
Argo CD status	Sync status: Synced Health status: Healthy.
Pods	Running with the correct image and replica count.
HTTP response	200 OK and application works.

Common Mistakes

- Reverting the wrong commit.
- Forgetting to push the change to Git.
- Leaving auto-sync enabled and losing the rollback.
- Not checking the actual image tag in the running pod.
- Skipping the application test after recovery.

Pro Tips

- Use meaningful commit messages.
- Keep image tags immutable and versioned.
- Enable image provenance and signing.
- Monitor with Argo CD notifications.
- Practice rollback in your lab environment.

Key Takeaway

- Recovery starts with Git, not the cluster.
- Understand the difference between revert, redeploy, and Argo CD rollback.
- Automated sync is powerful, but it can change rollback behavior.
- Always verify the application is working after recovery.

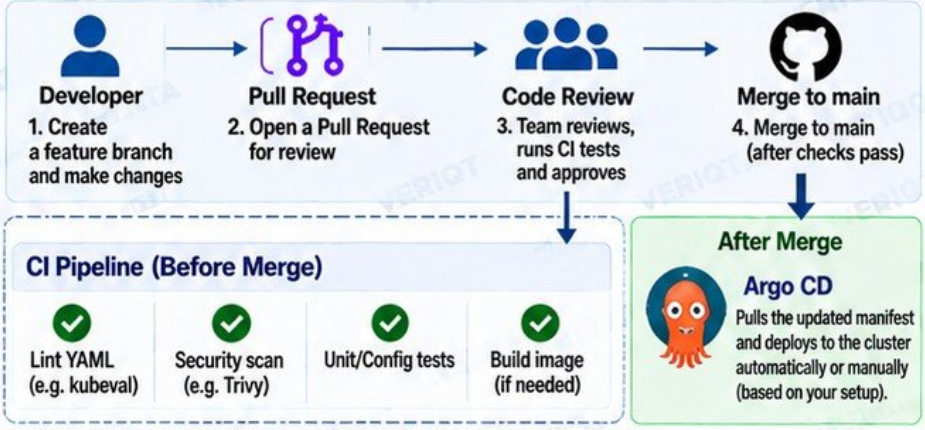
19. Before Production: The Controls Beginners Must Know

VERIQTA

Key Concepts

- Use Git branch protection to prevent unauthorized changes.
- Use pull requests (PR) and code review before merging.
- Apply least-privilege access for Git, Argo CD, and Kubernetes.
- Protect repository credentials and secrets.
- Use health probes to ensure application readiness and stability.
- Require deployment approvals for production changes.
- Separate development, staging, and production environments.
- Kubernetes Secrets stored as plain YAML are not automatically secure.

A Safer GitOps Workflow



1. Git Branch Protection

- Protect the main branch.
- Require pull requests for changes.
- Require at least one review.
- Block direct pushes.
- Require status checks (CI tests) to pass.

```
Settings → Branches → Add rule
✓ Require pull request before merging
✓ Require approvals
✓ Require status checks to pass
✓ Restrict who can push to matching branches
```

2. Pull Requests and Review

- Changes are proposed in a pull request.
- Team members review the changes.
- CI pipeline runs tests automatically.
- Only approved changes are merged.

Open a pull request [Create pull request]

- CI checks passed
- 1 approval required
- No merge conflicts
- Ready to merge

3. Least-Privilege Access

- Give each person only the access they need.
- Separate roles for developers, reviewers, and administrators.
- Use read-only access where possible.
- Use different credentials for Git, Argo CD and Kubernetes.

Role	Access
Developer	Read + create branches (open PRs)
Reviewer	Read + approve PRs
Argo CD	Read manifests (deploy only)
Kubernetes User	Namespace-level access only
Administrator	Full access (restricted to few people)

4. Protect Repository Credentials

- Use personal access tokens or GitHub Apps.
- Store credentials as encrypted secrets (e.g. in Argo CD or CI).
- Rotate credentials regularly.
- Do not hardcode tokens in manifests or scripts.

```
# Example: GitHub token stored as a secret
in Argo CD (not in plain text)
apiVersion: v1
kind: Secret
metadata:
  name: repo-credentials
type: Opaque
stringData:
  token: ${GITHUB_TOKEN}  # use Argo CD secrets
```

5. Health Probes

- Use readiness probe to check if the app is ready.
- Use liveness probe to restart unhealthy Pods.
- Prevent traffic to unhealthy containers.

```
readinessProbe:
  httpGet:
    path: /health
    port: 8080
  initialDelaySeconds: 5
  periodSeconds: 10
livenessProbe:
  httpGet:
    path: /health
    port: 8080
  initialDelaySeconds: 15
  periodSeconds: 20
```

6. Deployment Approvals

- Use manual sync for production.
- Require approvals before deploying to prod.
- Use a separate Argo CD project with restricted access.
- Consider using sync windows or change freeze during critical periods.

Argo CD Sync Policy (Production)

- (●) Manual (recommended for production)
- ( ) Automatic (use with approvals and controls)

7. Separate Environments

- Use different namespaces or clusters for dev, staging, and prod.
- Use separate Argo CD applications or projects.
- Apply different sync policies and access controls.

Environment	Namespace	Sync Policy	Access
Development	veriqta-dev	Automatic	Dev team
Staging	veriqta-staging	Automatic/Manual	Dev + QA
Production	veriqta-prod	Manual (with approval)	Restricted

8. Kubernetes Secrets Are Not Automatically Secure

- Secrets stored as plain YAML are only base64 encoded (not encrypted).
- Anyone with secrets to the repo can decode them.
- Use external secret managers or encrypted secrets in production.
- Restrict repo access and use controls like sealed-secrets, SOPS or a cloud secret manager.

Example: Secret in YAML (base64 only)

```
apiVersion: v1
kind: Secret
metadata:
  name: db-secret
type: Opaque
data:
  password: YWRtaW46MTIzNA== # base64
```

Key Takeaways

- Use Git branch protection, pull requests, and code review.
- Apply least-privilege access and protect credentials.
- Use health probes and approval controls.
- Separate environments and understand the limits of Kubernetes Secrets.

Common Mistakes

- Allowing direct pushes to main.
- Skipping code review and CI tests.
- Storing real secrets as plain YAML in Git.
- Giving excessive access to Git, Argo CD or the cluster.
- Using automatic sync in production without approvals.

Best Practices

- Keep manifests versioned in Git.
- Use automated tests and security scans.
- Use proper secret management.
- Start with manual sync for production.
- Document your GitOps workflow.

20. The Production Incident: Diagnose, Recover, and Prove It

VERIQTA

1. Incident Scenario

- A new application release causes intermittent HTTP 502 errors.
- One backend instance is restarting.
- The monitoring dashboard shows an increase in failed requests.

2. Architecture Overview



Failure can come from:
Git change | Argo CD sync | Kubernetes pods | Application config | Request path

3. Investigation Sequence

1. Confirm the current Git revision and deployed image.
2. Check Argo CD synchronization and health separately.
3. Inspect Deployment rollout, Pod restarts, events, and logs.
4. Check readiness and Service endpoints.
5. Compare the latest configuration with the known-good revision.
6. Choose a safe recovery action.
7. Verify that reconciliation completes.
8. Confirm application responses and recovery of failure metrics.

4. Check Git Revision and Deployed Image

```
# See latest commit
git log --oneline -5

# Check image in manifest
grep -i 'image:' k8s/deployment.yaml

# See deployed image in cluster
kubectl get deployment veriqta-demo \
 -n veriqta-demo -o jsonpath='{.spec.template
 .spec.containers[0].image}'

echo
```

5. Check Argo CD Status

```
# Application sync and health
argocd app get veriqta-demo

# Or through CLI shorter view
argocd app status veriqta-demo
# Check history
argocd app history veriqta-demo
```



veriqta-demo

- Sync Status: Synced
- Health Status: Degraded

6. Inspect Deployment and Pods

```
# Check rollout status
kubectl rollout status deployment/veriqta-demo \
 -n veriqta-demo
# View pods and restarts
kubectl get pods -n veriqta-demo -o wide
# Describe a failing pod
kubectl describe pod <pod-name> -n veriqta-demo
# Check recent events
kubectl get events -n veriqta-demo --sort-by='.lastTimestamp'
```

7. Check Application Logs

```
# View logs from the failing pod
kubectl logs <pod-name> -n veriqta-demo

# Follow logs in real time
kubectl logs -f <pod-name> \
 -n veriqta-demo
# If multiple containers
kubectl logs <pod-name> -c <container> \
 -n veriqta-demo
```

8. Check Readiness and Endpoints

```
# Check pod readiness
kubectl get pods -n veriqta-demo \
 -o wide
# Check Service endpoints
kubectl get endpoints -n veriqta-demo

# Test the service from inside cluster
kubectl run test --rm -it --image=busybox \
 -- sh -c 'wget -qO- http://veriqta-demo'
```

9. Compare with Known-Good Revision

```
# Show recent commits
git log --oneline -10

# View differences
git diff <good-commit>..<bad-commit>

# Check what changed (e.g. image tag, config, env)
git show <bad-commit>
```

10. Recovery Actions

- If the change introduced the issue, revert through Git.
- Allow Argo CD to reconcile automatically (or sync manually if auto-sync is off).
- Verify the rollout and application health before closing the incident.

```
# Revert the bad commit
git revert <bad-commit>
git push origin main
```

11. Verify Recovery

- Check Argo CD sync status and health.
- Confirm all replicas are running.
- Test application HTTP response.
- Check monitoring dashboard for normal metrics.

```
kubectl get pods -n veriqta-demo
curl -I http://<service-ip>:<port>
argocd app get veriqta-demo
```

12. Incident Report (Final Deliverable)

Write a short report with the following sections:

1. Timeline (what happened and when)
2. Evidence (commands, outputs, screenshots)
3. Root cause or remaining uncertainty
4. Recovery action (what you did)
5. Verification results (how you confirmed it works)
6. One prevention recommendation

13. Key Takeaways

- Argo CD can deploy the configuration Git declares, but it cannot guarantee that the application code is correct or that every production dependency is healthy.
- Check Git, Argo CD, Kubernetes, application logs, and the request path.
- Use a controlled and documented recovery process.
- Improve monitoring, testing, and review processes to reduce future incidents.

14. Common Mistakes

- Blaming Argo CD for application bugs.
- Not checking Git changes.
- Ignoring Pod events and logs.
- Only checking sync status (not health).
- Reverting in the cluster instead of Git.
- Not verifying that the application actually works after recovery.